Claude Certified Architect — Foundations

Руководство по подготовке от первых принципов

> Составлено на основе официального руководства к экзамену и банка из 200 практических вопросов. Концепции выстроены так, чтобы каждая новая идея опиралась только на уже изученные.

Как читать это руководство

Каждый раздел построен по одной и той же схеме:

1. **Первый принцип** — лежащая в основе истина, из которой вытекает правило.
2. **Что из него следует** — конкретные правила и шаблоны, которые проверяются на экзамене.
3. **Конкретные примеры** — как минимум один.
4. **Анти-паттерны** — что *выглядит* правильным, но таковым не является, и почему.

Если вам нужно запомнить только один мета-принцип, запомните этот:

> **Экзамен оценивает архитектурное мышление, а не запоминание терминов.** Каждый «правильный» ответ устраняет *первопричину*; каждый неправильный лечит *симптом*. Авторы экзамена помечают неверные ответы фразами вроде «*заманчивое короткое решение*», «*часто предлагаемая заплатка*», «*работает в простом случае и тихо ломается на сложных*». Тренируйтесь распознавать этот запах.

ЧАСТЬ 0 — Основания (читайте в первую очередь)

Это атомы. Всё остальное собрано из них.

0.1 Что такое LLM на самом деле, в одном абзаце

LLM (Large Language Model) — это функция, принимающая на вход **последовательность токенов** и выдающая **распределение вероятностей следующего токена**. Многократная выборка из этого распределения и порождает ответ. У модели нет памяти между вызовами, нет доступа к внешнему миру, нет понятия времени, нет встроенных инструментов. Каждый вызов не имеет состояния: всё, что модель «знает» о вашем разговоре — это то, что вы положили во вход. **Этот единственный факт — источник более чем половины правил данного экзамена.**

Если модель что-то «забыла» — вы не положили это во вход. Если модель «галлюцинировала» факт — вы не привязали её к источнику. Если модель «не следовала вашему правилу» — ваше правило живёт в прозе, а модель вероятностна.

0.2 Токены и почему контекст конечен

Токен — это кусок текста (примерно ¾ слова). Вход + выход вместе должны помещаться в **окно контекста** модели (например, 200K токенов). Каждый байт системного промпта, каждое описание инструмента, каждое предыдущее сообщение, каждый результат вызова инструмента отнимает место. Поэтому:

- Длинные результаты инструментов **крадут ёмкость** у последующих рассуждений.
- Многоходовые диалоги растут до тех пор, пока не вытеснят способность модели обращать внимание на ранние факты.
- «Просто возьмите окно побольше» редко бывает правильным ответом — это меняет размер комнаты, но не **качество внимания** внутри неё (см. «потеряно в середине», §6.1).

0.3 Что такое «инструмент»

Инструмент — это описание функции (имя, JSON-schema параметров, текстовое описание), которое вы отдаёте модели. Модель может «вызвать» инструмент, выдав структурированный вывод с именем инструмента и аргументами. **Ваш код** затем фактически выполняет функцию, фиксирует результат и возвращает его модели как сообщение. Модель никогда не выполняет ничего сама.

Поэтому «вызов инструмента» — это запрос, а не действие. Выполняете его вы. Возвращаете результат вы. Модель решает, что делать дальше.

0.4 Три уровня, которые проверяет экзамен

Уровень	Что это	Что вы строите с его помощью
Claude API	Низкоуровневый HTTP-эндпоинт: messages, tools, tool_choice, stop_reason	Что угодно кастомное
Claude Agent SDK	Библиотека поверх API, которая запускает агентный цикл, управляет суб-агентами (`Task`-инструмент) и поддерживает хуки	Продакшн-агенты (поддержка, исследовательские конвейеры)
Claude Code	CLI/IDE-приложение поверх Agent SDK, конфигурируется через `CLAUDE.md`, `.claude/`, MCP-серверы	Рабочие процессы разработчиков
MCP (Model Context Protocol)	Стандарт для предоставления инструментов и ресурсов модели от внешнего сервера	Интеграции с бэкендами

Эти уровни наслаиваются. Claude Code использует Agent SDK. Agent SDK использует API. MCP — это способ *публикации* инструментов, который любой из этих уровней может потреблять.

0.5 Агентный цикл — самое важное понятие на экзамене

«Агент» — это `while`-цикл, а не магический паттерн. Цикл такой:

```
loop:
    response = call_claude(messages, tools)
    if response.stop_reason == "end_turn":
        break  # модель закончила, возвращаем текст пользователю
    if response.stop_reason == "tool_use":
        for each tool_use in response:
            result = execute(tool_use)
            messages.append(tool_result)
        continue  # спрашиваем Claude что делать дальше с учётом новых результатов
```

И всё. Несколько вопросов экзамена строятся на правильном понимании этого, поэтому сразу назовём режимы отказа:

- **Неверно: завершать, если ответ содержит текст.** Модель может выдавать пояснительный текст *между* вызовами инструментов. Текст не означает «готово».
- **Неверно: завершать после N итераций как основной механизм.** Жёсткое ограничение нужно как *страховочный механизм*, но *основной* сигнал завершения — это `stop_reason == "end_turn"`.
- **Неверно: парсить естественный язык («Готово!» / «Закончено») для решения.** Вероятностно и хрупко.
- **Верно:** ветвление по `stop_reason`. `"tool_use"` → выполнить и зациклить. `"end_turn"` → остановиться. `"max_tokens"` → обработать (например, суммировать вывод инструмента и

продолжить или сообщить об усечении).

0.6 Самый глубокий принцип всего экзамена

> **Текстовые инструкции модели вероятностны. Код, который работает вокруг модели, детерминирован. Когда цена сбоя высока — переносите правило в код.**

Каждый вопрос вида «это должно быть в системном промпте, в описании инструмента или в хуке?» сводится к этому. Иерархия силы принуждения, от слабейшей к сильнейшей:

1. **Правило в системном промпте** («ты должен проверить перед возвратом денег») — вероятностное. Доля сбоев > 0.
2. **Few-shot пример** («вот пример правильного поведения») — вероятностное, чуть лучше.
3. **Описание инструмента** («вызывай только после X») — вероятностное, ограничено одним инструментом.
4. **Ограничение JSON-schema** («это поле — enum из {A,B,C}») — устраняет *класс* сбоев (некорректные значения).
5. **Конфигурация `tool_choice`** («следующий ответ должен вызвать инструмент») — устраняет «инструмент не был вызван».
6. **Хук PreToolUse / PostToolUse** — код, перехватывающий каждый релевантный вызов; может блокировать, преобразовывать, нормализовать. **Детерминированно.**
7. **Логика на уровне инструмента** — сам инструмент отвергает невалидный вход или обеспечивает идемпотентность.

Когда вопрос звучит «агент делает X в 2-12% случаев, хотя мы говорили не делать», ответ почти всегда — подняться по этой лестнице.

0.7 Почему «возьми модель побольше» почти никогда не правильный ответ

Шаблон экзамена: неверный ответ часто предлагает «переключиться на модель с большим окном контекста» или «использовать модель побольше / поумнее». Это неверно потому что:

- Большие окна не лечат **качество внимания** (потеряно в середине).
- Большие модели не превращают вероятностное принуждение в детерминированное.
- Большие модели не лечат расплывчатые критерии, отсутствие схем или устаревшие результаты инструментов.

Относитесь к «возьми модель побольше» так же, как к «подними температуру» или «добавь акцента в промпт»: почти всегда отвлекающий вариант.

0.8 Анатомия запроса к Messages API

Anthropic Messages API принимает массив `messages`, у каждой записи — `role` (`"user"` или `"assistant"`) и `content`. Содержимое — либо строка, либо массив типизированных **блоков**:

Тип блока	Где встречается	Назначение
`text`	оба	Простой текст
`image`	в основном user	Изображение base64, URL или file_id
`document`	user	PDF, передаваемый модели
`tool_use`	assistant	Модель вызывает инструмент (`id`, `name`, `input`)
`tool_result`	**user**	Ваш ответ на `tool_use` (`tool_use_id`, `content`, опц. `is_error`)
`thinking`	assistant	Вывод extended-thinking (несёт `signature`, который нельзя менять)

Поля верхнего уровня запроса: `model`, `max_tokens`, `system`, `tools`, `tool_choice`, `temperature`, `stop_sequences`, `stream`, `metadata`. Поля ответа: `id`, `role`, `content`, `stop_reason`, `stop_sequence`, `usage` (`input_tokens`, `output_tokens`, `cache_creation_input_tokens`, `cache_read_input_tokens`).

Два правила, которые часто пропускают:

1. **Tool results живут в user-сообщениях.** Когда ассистент выдал `tool_use`, ваш ответ имеет `role: "user"` с одним или несколькими `tool_result`-блоками. Поместить их в assistant-сообщение — это `400`.
2. **Воспроизведённые ходы ассистента должны сохранять полный порядок блоков и `signature` у `thinking`.** Нельзя редактировать прежний контент ассистента и пересылать. Подпись — проверка целостности; вмешательство возвращает `400` или молча портит рассуждения на следующем ходе.

0.9 Стриминг

Установите `stream: true`, и API вернёт Server-Sent Events. События, которые встретятся:

Событие	Смысл
`message_start`	Каркас ответа с пустым content
`content_block_start`	Открывается новый блок (`text` / `tool_use` / `thinking`)
`content_block_delta`	Инкрементальный контент; для `tool_use` это `input_json_delta` (частичный JSON)
`content_block_stop`	Блок завершён — содержимое теперь полное
`message_delta`	Финальные обновления: `stop_reason`, `usage`
`message_stop`	Конец стрима

Стриминг для tool_use: на `content_block_start` показывайте «Вызываю `search_database`...», накапливайте чанки `input_json_delta`, **исполняйте только на `content_block_stop`**. Не пытайтесь

парсить частичный JSON — он невалиден до закрытия блока.

0.10 Что меняется между поколениями моделей Claude

Внутри Claude 4.x:

- **Opus** — самые тяжёлые рассуждения, самая способная агентная работа, наивысшая стоимость.
- **Sonnet** — сбалансированный продакшн-дефолт.
- **Haiku** — дёшево и быстро: роутинг инструментов, классификация, извлечение.

Возможности (extended thinking, vision, PDF, prompt caching, batch, citations, tool use) стабильны в линейке 4.x. Никогда не зашивайте проверки возможностей по фиксированной строке модели в продакшене. Когда экзаменационный вопрос предлагает «переключиться на модель побольше», считайте это отвлекающим вариантом — если только речь не о *возможности*, а не о *качестве*.

ЧАСТЬ 1 — Домен 1: Агентная архитектура и оркестрация (27% экзамена)

Это самый большой домен. Он о том, как корректно построить цикл и как безопасно компоновать несколько агентов.

1.1 Агентный цикл подробно

Уже введён в §0.5. Экзамен раз за разом проверяет три нюанса:

1.1.1 Stop reasons — исчерпывающая таблица

`stop_reason`	Что означает	Что делает ваш цикл
`tool_use`	Модель хочет вызвать один или несколько инструментов	Выполнить их, добавить `tool_result` сообщения, зациклить
`end_turn`	Модель считает, что закончила	Вернуть текст пользователю, выйти из цикла
`max_tokens`	Модель упёрлась в лимит токенов на середине ответа	**Обработать явно** — обычно суммировать частичный вывод и продолжить или сообщить об усечении. Не считать это `end_turn`.
`stop_sequence`	Совпадение с заданной стоп-последовательностью	Зависит от приложения

Вопрос-ловушка: инструмент возвращает огромный результат. Следующий вызов модели упирается в `max_tokens`. Цикл написан как `while stop_reason !== "end_turn"`, поэтому он молча завершается без ответа пользователю. Правильное исправление: **обнаружить `max_tokens` и обработать его**.

1.1.2 Лимит итераций — страховочный механизм

У каждого цикла должен быть жёсткий лимит (например, 25 итераций). Парьте лимит с фолбэком (передать человеку, вернуть частичный результат с пометкой «не сошлось»). Лимит — это не *основной* механизм завершения, его роль — `stop_reason`. Лимит — подушка безопасности.

1.1.3 Ошибки инструментов — нормальный вход цикла

Если инструмент возвращает ошибку, агент читает её как `tool_result`, решает, что делать (повторить с другими аргументами, попробовать другой инструмент, сдаться и сообщить), и цикл продолжается. **Ошибки инструмента не должны завершать цикл.** Они завершают *вызов инструмента*.

1.2 Хуки — слой детерминированного принуждения

Agent SDK предоставляет хуки:

- **`PreToolUse` хук** — выполняется до того, как вызов инструмента будет исполнен. Может проверить предлагаемый вызов и **заблокировать**, изменить его или пропустить.
- **`PostToolUse` хук** — выполняется после возврата вызова инструмента. Может преобразовать результат до того, как модель его увидит.

Хуки — это обычный код. Они детерминированы.

1.2.1 Для чего нужен `PreToolUse`

- **Шлюзы предусловий.** Блокировать `process_refund` пока в этой сессии не завершился `get_customer`.
- **Принудительное соблюдение политик.** Отклонять любой `process_refund` где `amount > 500`; перенаправлять на эскалацию.
- **Стены подтверждения.** Отклонять любой денежный вызов, у которого нет соответствующего токена подтверждения от пользователя в разговоре.
- **Проверки состояния сессии.** «Был ли вызов search-tickets перед create-ticket?»

1.2.2 Для чего нужен `PostToolUse`

- **Нормализация данных.** Три бэкенда возвращают временные метки в трёх форматах; хук переписывает их все в ISO 8601 до того, как модель их увидит. (Модели никогда не приходится конвертировать, поэтому она никогда не сравнивает их неправильно.)
- **Обрезка вывода.** Инструмент возвращает 60 полей, важны 4; хук выбрасывает остальные.
- **Аудит-логирование.** Эмиттить структурированную телеметрию для каждого вызова инструмента.
- **Связывание рабочих процессов.** После каждого `Edit` автоматически запускать тесты.

1.2.3 Почему это важнее любого другого урока

Многие вопросы экзамена описывают «у нас 2% сбоев и мы хотим 0». Если правило проверяемо по данным, которые система уже имеет (аргументы вызова, история сессии, ответ инструмента), ответ — **хук**, а не более сильный промпт. Промпты вероятностны; хуки нет.

1.3 Многоагентная оркестрация

Координатор — это верхнеуровневый агент, в чей набор инструментов входит `Task` (порождает суб-агентов). **Суб-агенты** — это независимые разговоры Claude, каждый из которых выполняет конкретную специализированную роль (веб-поиск, анализ документов, синтез, генерация отчётов и т. д.).

1.3.1 Два фундаментальных факта о суб-агентах

1. **Суб-агенты не наследуют контекст координатора.** Они начинают с того, что вы поместили в их промпт. Если вы порождаете суб-агента «исследовать ИИ в здравоохранении» и даёте ему лишь строку «Research AI in healthcare», он понятия не имеет, что на самом деле нужно пользователю, что уже найдено и какие подвопросы остались. Он выдаст поверхностный общий вывод. **Решение: явно передавать полный контекст (цель, область, предыдущие находки, ожидаемая форма вывода) в промпте суб-агента.**
2. **У координатора `"Task"` должен быть в `allowedTools`.** Без него он не может порождать суб-агентов. (Невидимый вопрос про настройку.)

1.3.2 Параллельный запуск суб-агентов

Чтобы запускать суб-агентов параллельно, координатор испускает **несколько вызовов инструмента `Task` в одном ответном ходе**. Agent SDK выполнит их параллельно. Чтобы выполнить их последовательно — испускайте по одному `Task`-вызову на ход и ждите. Последовательный режим корректен только когда есть реальная зависимость по данным (суб-агенту B нужен вывод суб-агента A).

> Ловушка: «Как мне распараллелить?» Неверные варианты включают `async: true` (выдуманный), `fork_session` (другая цель, см. §1.6), middleware (переусложнение). Верный ответ: **несколько `Task`-вызовов в одном ответе.**

1.3.3 Динамическая маршрутизация координатора

Координатор, всегда вызывающий весь конвейер на каждый запрос, расточителен. Правильно спроектированный координатор **оценивает сложность запроса** и выбирает, каких суб-агентов запускать. Простой фактический запрос — может быть, только веб-поиск. Глубокое исследование — полный конвейер. (Подгоняйте процесс под работу.)

1.3.4 Декомпозиция координатора — место, где ломается покрытие

Распространённый сценарий экзамена: «Отчёты покрывают только подмножество X темы». Каждый суб-агент корректно отработал в рамках своей задачи. Координатор слишком узко декомпозировал тему. **Исправление — в координаторе, не в суб-агентах.** Это и есть смысл официального примера 7.

Надёжный координатор запускает **цикл итеративного уточнения**: после синтеза проверить покрытие; если есть пробелы — повторно делегировать поиск/анализ с целевыми запросами; пере-синтезировать. Повторять, пока критерии покрытия не выполнены.

1.3.5 Маршрутизация всех коммуникаций через координатор

Суб-агенты не должны общаться друг с другом напрямую. Всё проходит через координатор. Почему?

- **Наблюдаемость.** Одно место для логов, одно место для отладки.

- **Согласованная обработка ошибок.** Координатор — единственная точка решений по повторам, фолбэкам, приёмке частичных результатов.
- **Контроль информации.** Координатор решает, что видит каждый суб-агент, предотвращая утечку контекста и непреднамеренные зависимости.

1.3.6 Дисциплина по числу инструментов (связь с §2.3)

Каждый суб-агент получает только нужные ему инструменты. Агент синтеза не получает веб-поиск. (Иначе он будет выходить за рамки специализации и выдавать плохие результаты.) Исключение: чётко ограниченный помощник для частой нужды — OK. Например, инструмент `verify_fact` для агента синтеза, где 85% верификаций — простой поиск, и только 15% требуют полного делегирования через координатор.

1.4 Передача контекста суб-агентам

Два факта:

- Промпт суб-агента должен содержать всё, что ему нужно.
- Для провенанса **передавайте структурированные данные**, а не прозу. Находка должна быть `{claim, evidence_excerpt, source_url, publication_date}`, а не «Smith сказал X».

1.4.1 Включение находок предыдущих агентов

Если агенту синтеза нужны результаты веб-поиска и вывод анализа документов, **вставьте их в промпт агента синтеза напрямую**. Автоматического наследования нет. (Одно из учебных упражнений в руководстве — именно это.)

1.5 Шаблоны многошаговых рабочих процессов

1.5.1 Программные предусловия (канонический пример)

> Продакшн-данные: в 8% случаев агент возвращает деньги, не верифицировав личность клиента. Что делать?

- **Неверно: усилить формулировку промпта.** Вероятностно; 8% превратится в 6%.
- **Неверно: `tool_choice` с принуждением выбора на каждый ход.** Не та гранулярность — агент не должен быть *вынужден* верифицировать, он должен быть *заблокирован* от возврата денег без верификации.
- **Неверно: классификатор маршрутизации.** Решает другую задачу (какие инструменты выставлять), а не порядок.
- **Верно: `PreToolUse`-хук на `process_refund`, который проваливает вызов, если в этой сессии не завершён верифицированный `get_customer`.** Детерминированно. Агент

свободно может вызывать возврат денег; он просто *не может* до того, как клиент верифицирован.

1.5.2 Структурированная передача людям

Когда агент эскалирует, человек-обозреватель не видит разговора. Ему нужна **структурированная сводка**: ID клиента, корневая причина, рекомендуемое действие, сумма возврата (если есть). Не полный транскрипт (длинно, без синтеза), не просто email (теряется итог).

Качество передачи (включила ли она эти поля?) — то, что предсказывает скорость решения вопроса человеком. Это и есть правильная метрика успешности handoff'a.

1.5.3 Декомпозиция многотемных запросов

Пользователь пишет о трёх проблемах в одном сообщении. Неверно: обрабатывать их поочерёдно, смешивая контекст. Верно: декомпозировать на отдельные пункты, расследовать параллельно с общим контекстом, синтезировать единое решение.

1.6 Управление сессиями

1.6.1 Четыре рычага

- `--resume <session-name>` — продолжить именованную сессию. Использовать, когда предыдущий контекст в основном ещё актуален.
- `fork_session` — независимо разветвиться от общей базы. Использовать для сравнения двух подходов из одной отправной точки.
- `/compact` — сжать текущую сессию на месте. Использовать, когда **контекст велик, но факты ещё актуальны** (нет изменений кода, нет устаревшего состояния).
- **Начать заново + ввести сводку** — полностью отказаться от сессии. Использовать, когда **предыдущие результаты инструментов устарели** (код изменился, мир сдвинулся). Возобновление с устаревшими результатами лишь распространяет устаревание.

1.6.2 Вопрос-ловушка

> Вы вчера завершили длинное расследование. Сегодня в репозитории значительные изменения. Вы возобновляете — агент ссылается на функции, которых уже нет. Лучший следующий шаг?

- `/compact` неверно: сжимает устаревание, не убирает его.
- Перечитать каждый файл неверно: не устраняет структурное устаревание в предыдущих рассуждениях модели.
- Промпт-напоминание «код мог измениться» неверно: вероятностная заплатка на структурную проблему.
- **Верно: начать новую сессию со структурированной сводкой предыдущих находок.** Выбросить устаревшие результаты инструментов.

1.6.3 Сценарии `fork_session`

Три «брата» из одной базы:

- «Сравнить две стратегии рефакторинга, исходя из того же анализа.»
- «Попробовать три варианта миграции.»
- «A/B-тест двух подходов к промптам на одном и том же контексте.»

1.7 Восстановление от ошибок в многоагентных системах

Если суб-агент упал, координатор должен:

- Получить **структурированную ошибку** (тип сбоя, что было предпринято, частичные результаты, альтернативы) — **а не** обобщённое «search unavailable».
- Решить: повторить с другими аргументами, попробовать альтернативного суб-агента, принять частичные результаты.
- **Аннотировать финальный вывод** указанием, какие разделы хорошо покрыты, а где есть пробелы из-за недоступных источников.

Неверные шаблоны:

- Уронить весь конвейер на одном сбое.
- Молча вернуть пустой результат, выдав за успех.
- Перезапускать всех 8 суб-агентов, когда упали только 3 (повторяет успешную работу).

Когда упали 3 из 8 параллельных суб-агентов, **отслеживайте успех попо-агентно (с помощью идентификаторов в стиле `custom_id`) и повторяйте только сбойные с экспоненциальной выдержкой.**

1.8 Идемпотентность и двухфазные шаблоны для необратимых действий

Некоторые действия необратимы (отмена подписки, отправка письма, возврат денег). Агент иногда повторяет их, когда пользователь перефразирует («да, верни деньги» / «пожалуйста, оформи возврат»).

- **Ключи идемпотентности** на уровне инструмента предотвращают двойное выполнение.
- **Двухфазные инструменты**: `preview_cancel` возвращает потенциальный эффект и токен подтверждения; `confirm_cancel` требует этот токен. Структурно дважды нажать нельзя.

Это правки на уровне инструментов, а не правки промпта.

1.9 Суб-агент Explore

В Claude Code есть встроенный суб-агент Explore для подробного исследования (картирование большой кодовой базы, трассировка зависимостей). Он работает в изоляции и **возвращает сводку**.

Анти-паттерн: заставить Explore вернуть листинг на 4000 строк в основную сессию — это хуже, чем не использовать Explore вовсе. **Всегда просите суб-агента суммировать.**

1.10 Уверенность некалиброванна

Сгенерированные LLM оценки уверенности коррелируют с многословностью, а не с правильностью. Не делайте решения зависимыми от поля `confidence`, которое модель сама о себе выдаёт.

Используйте **структурные проверки**: сходится ли арифметика? Существует ли URL источника? Проходит ли схема валидацию? Завершился ли предусловный шаг? Это детерминированные сигналы.

1.11 Повторяющиеся анти-паттерны Домена 1, которые надо усвоить

- «Усилить формулировку в системном промпте» — прозовая заплатка на структурную проблему.
- «Добавить few-shot примеры» — полезно для *неоднозначности*, не для *принуждения*.
- «Уменьшить температуру» — против креативного дрейфа, не против логики цикла, не против принуждения, не против покрытия.
- «Окно побольше / модель поумнее» — не лечит ни качество внимания, ни декомпозицию, ни выбор «промпт vs хук».
- «Сказать агенту быть осторожнее» — никогда не ответ.

1.12 Полная таксономия хуков (Agent SDK)

Помимо `PreToolUse` и `PostToolUse`, Agent SDK выпускает хуки на каждом интересном переходе. Они один к одному соответствуют событиям хуков Claude Code (§3.11):

Событие	Срабатывает	Для чего
`PreToolUse`	Перед каждым вызовом инструмента	Блокировка, изменение, шлюз предусловий
`PostToolUse`	После каждого результата	Нормализация, обрезка, лог, цепочка
`UserPromptSubmit`	На каждое новое сообщение пользователя	Впрыснуть контекст, очистить PII, прицепить «case-facts» блок (§5.3)
`Stop`	Цикл вот-вот завершится (`end_turn`)	Финальная QA-проверка, сохранить scratchpad
`SubagentStop`	Суб-агент закончил	Проверить форму вывода; отвергнуть и переделегировать
`PreCompact`	До запуска `/compact`	Закрепить факты, чтобы они пережили компактификацию
`SessionStart`	Сессия открывается	Загрузить case-facts файл, выставить дефолты, прогреть кеши
`SessionEnd`	Сессия закрывается	Слить телеметрию, синхронизировать scratchpad
`Notification`	Сайд-канальное сообщение пользователю	Кастомные оповещения / интеграции

Распространённая ловушка экзамена: «Как впрыснуть дополнительный контекст в каждое сообщение пользователя?» Ответ: **`UserPromptSubmit`-хук**, а не «изменить системный промпт» или «попросить пользователя вставить вручную». Симметрично: «Как сделать, чтобы заметки scratchpad пережили `/compact`?» Ответ: **`PreCompact`-хук**, который перепинит их.

1.13 Файлы определений суб-агентов (`.claude/agents/`)

Отличаются от скиллов (§3.4). Файлы суб-агентов лежат в `.claude/agents/<name>.md` и определяют **делегируемого специалиста**, на которого координатор может нацелить инструмент `Task` по имени.

```
---
name: code-reviewer
description: Review pending changes for bugs and security issues
tools: ["Read", "Grep", "Glob", "Bash(git diff:*)"]
model: claude-haiku-4-5
---
You are a code reviewer. Focus on logic bugs, security issues, and breaking API
changes. Skip style. Return findings as a JSON array of {file, line, severity, issue}.
```

Поле	Эффект
`name`	Идентификатор, по которому координатор порождает суб-агента
`description`	Что видит координатор, решая кому делегировать
`tools`	Allowlist инструментов (подмножество родительского, грамматика разрешений §3.10)
`model`	Опциональное переопределение — Haiku для дешёвых классификаторов, Opus для тяжёлого анализа

Скилл vs суб-агент vs slash-команда — выбираем одно:

Что нужно	Использовать
Детерминированную пошаговую процедуру (Claude читает markdown и следует)	**Скилл**
Независимое расследование, возвращающее сводку	**Суб-агент** (через `Task`)
Переиспользуемый шаблон промпта, вызываемый пользователем	**Slash-команда**

Скилл по умолчанию работает в той же сессии; суб-агент всегда — в изоляции; slash-команда — это просто шаблон промпта. Путать их — частый шаблон неверного ответа.

1.14 Thinking-блоки в цикле

Когда extended thinking включён, ответ ассистента может содержать `thinking`-блоки наряду с `text` и `tool_use`. **Вы должны сохранять их — включая поле `signature` — дословно при воспроизведении хода ассистента обратно в API.** Удаление или изменение либо возвращает `400`, либо молча ухудшает рассуждения на следующем ходе, потому что цепочка размышлений сломана.

Практические правила:

- Массив `messages` в цикле хранит `thinking`-блоки наряду со всем остальным — не отфильтровывайте их.
- Не суммируйте, не редактируйте и не переформатируйте содержимое thinking перед пересылкой.
- Не показывайте сырое thinking конечным пользователям — храните на сервере.
- **Interleaved thinking** (Sonnet/Opus 4.x): модель может думать *между* вызовами инструментов в одном ходе. То же правило сохранения применимо — у каждого `thinking`-блока остаётся подпись.

1.15 Стриминг частичного входа инструмента

При стриминге вход tool_use приходит как последовательность `input_json_delta`-чанков. Правильный шаблон:

1. На `content_block_start` для `tool_use`-блока показать заглушку «инструмент стартует» (имя `name` уже известно).
2. Накапливать `input_json_delta`-чанки — пока **не** парсить.
3. На `content_block_stop` JSON полный. Теперь исполнять.

Не пытайтесь парсить частичный JSON. Он невалиден до закрытия блока.

ЧАСТЬ 2 — Домен 2: Дизайн инструментов и интеграция MCP (18% экзамена)

2.1 Что такое MCP, в одном абзаце

Model Context Protocol — стандарт предоставления модели от внешнего сервера двух вещей: **инструментов** (действия, которые модель может вызывать) и **ресурсов** (пассивный контент, который модель может видеть, например схема или каталог). MCP-серверы запускаются как локальные процессы или удалённые сервисы. Claude Code и Agent SDK подключаются к MCP-серверам и обнаруживают их инструменты/ресурсы при запуске. Модель затем видит эти инструменты наряду со встроенными.

Смысл MCP — **стандартизация**: вместо написания кастомной интеграции для каждого бэкенда вы реализуете протокол MCP однажды, и любой MCP-совместимый клиент может им пользоваться.

2.2 Описания инструментов — это то, как модель выбирает инструмент

Модель выбирает инструменты на основе (в порядке важности):

1. **Описание** инструмента.
2. **Имя** инструмента.
3. **Схема параметров** инструмента.
4. Примеры в системном промпте.

Два инструмента с описаниями вроде «Возвращает информацию о клиенте» и «Возвращает детали заказа» будут постоянно путаться. Решение — **конкретность**:

> «Возвращает историю выставленных счетов клиента за последние 90 дней. Использовать, когда пользователь спрашивает о прошлых списаниях или недавних счетах. Вход: `customer_id` (формат: `C-NNNNN`). Возвращает записи о биллинге, отсортированные по убыванию даты.»

Хорошее описание указывает **что, когда, зачем, ожидаемые входы, ожидаемые выходы**. Самореклама («ИСПОЛЬЗУЙ ЭТОТ ИНСТРУМЕНТ ВСЕГДА») искажает выбор — она не помогает.

2.3 Правило количества инструментов

Агент с 4–5 хорошо описанными, релевантными роли инструментами выбирает надёжно. Агент с 18 инструментами выбирает ненадёжно. Сложность решения растёт хуже, чем линейно. **Сокращайте безжалостно.** Это канонический вопрос про количество инструментов (D2-001).

2.4 Разделение универсальных инструментов

Один `analyze_document`, используемый для извлечения, суммирования и верификации, будет применяться неправильно. Разделите его:

- `extract_data_points`
- `summarize_content`
- `verify_claim_against_source`

У каждого одна цель, чистая схема, недвусмысленное описание. Агент сможет выбирать корректно, потому что инструменты больше не пересекаются.

2.5 Именование и устранение неоднозначности

Если два MCP-сервера выставляют инструмент с именем `search`, добавьте пространства имён или переименуйте. Модель видит их как «search» и «search» — коллизия структурная, не описательная.

2.6 Структурированные ответы об ошибках

Каждый ответ MCP-инструмента об ошибке должен включать:

- **`isError: true`** (стандартный флаг MCP)
- **`errorCategory`**: одно из `transient | validation | permission | business`
- **`isRetryable`**: булево (явно, не выводимо)
- **Человекочитаемое сообщение**

Зачем: агент использует эти метаданные для решения, что делать.

- `transient` + `isRetryable: true` → агент повторяет (возможно, с выдержкой)
- `validation` → агент исправляет вход
- `business` (например, нарушение политики, слишком большой возврат) → агент объясняет пользователю и эскалирует
- `permission` + `isRetryable: false` → агент эскалирует без повтора

Универсальное `{"status": "error", "message": "Operation failed"}` — канонически плохой ответ — у агента нет информации для восстановления.

2.7 Пустые результаты vs сбои

Если инструмент возвращает `[]` и для «нет совпадений», и для «бэкенд недоступен», вы потеряли критическое различие. Агент будет считать сбой «успешно ничего не нашли». **Всегда возвращайте структурированную ошибку для сбоев, отличную от валидного пустого результата.**

2.8 `tool_choice`

Три значения:

Значение	Смысл
`"auto"` (по умолчанию)	Модель может вызвать инструмент **или** вернуть текст
`"any"`	Модель **должна** вызвать какой-то инструмент, выбирает сама
`{"type": "tool", "name": "X"}`	Модель **должна** вызвать конкретно инструмент X

Значения `"required"` нет. (Отвлекающий вариант-ловушка.)

Когда что использовать:

- `"auto"` — обычный разговорный поток.
- `"any"` — когда следующий ответ всегда должен быть вызовом инструмента (например, конвейер извлечения, который никогда не должен возвращать разговорный текст).
- Принудительный конкретный инструмент — когда вы знаете, какая схема извлечения применяется (например, вы уже классифицировали документ как счёт-фактуру).

Важно: `"any"` не лечит плохие описания. Модель всё равно выбирает по именам и описаниям; она просто *обязана* выбрать.

2.9 Скоупинг MCP-серверов

Два места:

- `.mcp.json` в корне проекта — **общий через систему контроля версий**. Для команда-широких MCP-серверов (общий GitHub-сервер компании, общий Jira-сервер).
- `~/.claude.json` в домашнем каталоге пользователя — **личный**. Для экспериментальных серверов и личных предпочтений.

Не коммитьте личные экспериментальные инструменты в `.mcp.json` — они засоряют командный каталог инструментов и увеличивают неоднозначность выбора у всех.

2.10 Секреты через подстановку переменных окружения

`.mcp.json` поддерживает подстановку `${ENV_VAR}`. Так:

```
{
  "github": {
    "command": "github-mcp",
    "env": {
      "GITHUB_TOKEN": "${GITHUB_TOKEN}"
    }
  }
}
```

Файл коммитится чисто; секрет живёт в окружении каждого разработчика.

2.11 Ресурсы vs инструменты

	Инструмент	Ресурс
Назначение	**Действие**, которое модель совершает	**Окружающий контент**, который модель видит
Примеры	`process_refund`, `search_web`, `read_file`	«список доступных таблиц БД», «иерархия документации», «сводки задач»

Если агент сейчас вызывает `list_topics` в начале каждой сессии, чтобы выяснить, что доступно — это знак, что данные должны быть **ресурсом**, а не инструментом. Ресурсы исключают исследовательские вызовы инструментов.

2.12 Встроенные инструменты (Claude Code)

Инструмент	Назначение
Read	Загрузить полное содержимое файла
Write	Создать/перезаписать файл
Edit	Точечная модификация по точному совпадению якоря
Bash	Выполнить shell-команду
Grep	Поиск *содержимого* файлов по шаблону (имена функций, строки ошибок)
Glob	Поиск *путей* файлов по шаблону (`**/*.test.tsx`)

Частые ошибки выбора:

- Прочитать каждый файл, чтобы «найти» что-то → должен быть Grep.
- Перечислить файлы, чтобы найти подходящие шаблону → должен быть Glob.
- Пытаться Edit, когда якорь-текст не уникален → откатиться к Read + Write.

Правильный способ исследовать незнакомую кодовую базу — **инкрементальный**: Grep по точкам входа, Read для следования за импортами, трассировать потоки оттуда. **Никогда не загружайте все файлы заранее.**

2.13 Прочие правила дизайна инструментов из банка

- **Валидируйте входы строго на уровне схемы.** Если `customer_id` должен соответствовать `C-\d{5}`, валидируйте. Отвергайте несовпадения структурированной ошибкой с указанием формата. Это не даст агенту передавать имя там, где нужен ID.
- **Предпочитайте опциональные/nullable поля принуждённым строкам.** Обязательное поле `email` принуждает к фабрикации, когда email неизвестен. Сделайте его nullable, чтобы агент мог спросить пользователя.

- **Замените универсальный `run_shell` на специализированные инструменты** — `run_tests`, `run_linter`, `format_file`. Уменьшает радиус поражения, не теряя пользы.
- **Ограничьте `http_fetch`-подобные инструменты** allowlist хостов; иначе риск SSRF.
- **Версионированные имена инструментов** для несовместимых изменений; держите аддитивные изменения (новые опциональные параметры) под тем же именем.
- **Прогрессивное раскрытие**: если у инструмента огромный ответ, верните `{summary, reference_id}` и предоставьте инструмент-компаньон `get_full(reference_id)`, который агент может вызвать при необходимости.
- **Разделение sync-fast / async-slow** для инструментов с сильно меняющейся задержкой бэкенда: `do_thing` (быстро возвращает или падает) и `do_thing_async` (возвращает job ID; компаньон `check_status`).
- **Подсказки о цене** — в **описании инструмента**, не в системном промпте — так они применяются везде, где инструмент выставлен.
- **Build vs buy**: предпочитайте существующие сообществом MCP-серверы для стандартных интеграций (GitHub, Jira). Стройте кастомные только под специфические командные процессы.
- **Обрезайте вывод инструментов в источнике** — добавление параметра `fields: [...]` или предоставление slim-варианта (`get_customer_summary`) дешевле, чем пост-фильтр в каждом агенте.

2.14 Транспорты MCP

Три способа, которыми MCP-клиент подключается к серверу:

Транспорт	Протокол	Когда использовать
`stdio`	Подпроцесс с пайпами	Локальные инструменты, single-user dev. Дефолт для `claude mcp add`.
`sse` (Server-Sent Events)	HTTP + SSE-стрим	Удалённые/общие серверы. Долгоживущий.
`http` (Streamable HTTP)	HTTP, request/response или стрим	Современный удалённый дефолт; поддерживает stateless-режим и возобновление

Конфигурация в `.mcp.json`:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"]
    },
    "internal": {
      "url": "https://mcp.example.com/",
      "transport": "http"
    }
  }
}
```

Ловушка: «Как поделиться сервером с командой?» Транспорт не решает шеринг — это делает *расположение конфига*. `.mcp.json` (закоммичен) шерится; `~/.claude.json` (личный) — нет (§2.9).

2.15 Третий примитив MCP: prompts

Помимо инструментов и ресурсов, MCP-серверы могут выставлять **prompts** — параметризованные шаблоны промптов, вызываемые пользователем. В Claude Code они появляются как slash-команды с именем `/mcp__<server>__<prompt>`. Используйте, когда переиспользуемая единица — *сам промпт* (например, `/mcp__jira__create_ticket_from_thread`), а не действие (это инструмент) и не окружающие данные (это ресурс).

Три примитива, одно правило выбора:

Что должен сделать пользователь	Выставить как
Вызвать переиспользуемый шаблон промпта	**Prompt** (slash-команда)
Чтобы модель совершила действие	**Tool**
Чтобы модель видела окружающий контент	**Resource**

2.16 Stateful vs stateless MCP-серверы

Сервер **stateful**, если сессия устанавливает состояние на сервере (открытые дескрипторы файлов, незавершённые транзакции). Stateful-серверы нельзя наивно балансировать; повторное подключение требует session resumption.

Сервер **stateless**, если каждый запрос самостоятелен. Stateless-серверы горизонтально масштабируются, переживают failover и являются правильным дефолтом для продакшена. Выносите состояние в БД или внешнее хранилище, а не в процесс сервера.

2.17 Roots и sampling (продвинутый MCP)

- **Roots** — клиент сообщает серверу, какие каталоги пользователь авторизовал. Сервер не должен предполагать доступ за пределы заявленных roots.
- **Sampling** — сервер может попросить клиента выполнить LLM-вызов от его имени. Полезно для серверов, которым нужен LLM, но которые не должны нести собственный API-ключ. Тестируется реже, но появляется как отвлекающий вариант с именованием.

ЧАСТЬ 3 — Домен 3: Конфигурация и процессы Claude Code (20% экзамена)

Это ваш самый слабый домен по результатам пробного экзамена. Замедляйтесь на каждом вопросе по Домену 3.

3.1 Иерархия CLAUDE.md

Существуют три уровня, и они **все наслаиваются** (ни один не перекрывает другой):

Расположение	Область	Использование
`~/.claude/CLAUDE.md`	**Пользовательская** — только для этого человека, не шерится через git	Личные предпочтения (например, «я люблю подробные объяснения»)
`<project-root>/CLAUDE.md` или `.claude/CLAUDE.md`	**Проектная** — шерится через систему контроля версий	Командные стандарты кодинга, проект-широкие конвенции
`<subdir>/CLAUDE.md`	**Каталоговая** — применяется при редактировании файлов в этом подкаталоге	Каталого-специфические шаблоны (например, `frontend/CLAUDE.md`)

Наслаивание, не перекрытие. Когда Claude редактирует файл в `src/components/`, загружаются root + `src/components/CLAUDE.md` + пользовательский. Они компонуются. Если конфликтуют — разрешайте в содержимом; не ждите, что одно подавит другое.

3.1.1 Классическая ошибка иерархии (D3-001)

Новый член команды не получает командных стандартов. Расследование показывает, что стандарты живут в `~/.claude/CLAUDE.md` на машинах существующих разработчиков. **Решение: переместить их в `<project>/CLAUDE.md`, чтобы они шли с репозиторием.** Пользовательские конфиги не шерятся.

3.1.2 Команда `/memory`

Используйте `/memory` для проверки, какие memory-файлы загружены в данный момент. Диагностический инструмент, когда поведение непоследовательно.

3.1.3 Когда разделять CLAUDE.md

Сфокусированный CLAUDE.md на 200 строк — нормально. **Разделяйте, когда он становится большим и многотемным** (например, 900+ строк, покрывающих API-конвенции, тестирование,

шаблоны БД, деплой). Разделите на `.claude/rules/<topic>.md` и ссылайтесь на них из CLAUDE.md через `@import`.

Не модуляризуйте превентивно — сложность должна оправдывать разделение.

3.2 `.claude/rules/` с glob-путями

Файлы правил в `.claude/rules/` поддерживают YAML frontmatter:

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---
# Конвенции тестирования
- Использовать блоки `describe`/`it`
- Использовать `userEvent`, не `fireEvent`
- ...
```

Правило **загружается только когда Claude редактирует подходящий файл**. Это правильный инструмент когда:

- Конвенция распространяется на несколько каталогов (тестовые файлы по всей кодовой базе)
- Правила применяются по *типу* файла, а не по каталогу
- Вы хотите экономить токены, не загружая нерелевантные правила

Когда использовать правила vs subdir CLAUDE.md:

- **Правила с glob-путями** → конвенции, привязанные к *типу/шаблону* файла, разбросанные по каталогам
- **Subdir CLAUDE.md** → конвенции, привязанные к *одному конкретному поддереву каталогов*

Если ваши тестовые файлы разбросаны (например, `Button.test.tsx` рядом с `Button.tsx`), subdir CLAUDE.md до них не достанет. Файл правил с `paths: ["**/*.test.*"]` достанет.

3.3 `@import` для модульного CLAUDE.md

```
# Стандарты проекта

@import .claude/rules/testing.md
@import .claude/rules/api-conventions.md
@import .claude/rules/deployment.md
```

Корневой файл остаётся пробегаемым индексом; детали живут рядом со своей темой.

3.4 Slash-команды vs скиллы

И то и другое — переиспользуемые workflow по запросу, и то и другое — в `.claude/`, и то и другое может быть проектным (закоммиченным) или пользовательским.

	Slash-команда	Скилл
Расположение	`.claude/commands/` (проект) или `~/.claude/commands/` (пользователь)	`.claude/skills/` (проект) или `~/.claude/skills/` (пользователь)
Файл	Markdown-файл с шаблоном промпта	Файл `SKILL.md` с frontmatter + телом
Frontmatter	Минимальный	`context: fork`, `allowed-tools`, `argument-hint`
Лучше всего для	Быстрых переиспользуемых промптов (`/review`, `/impact`)	Многошаговых процессов с дисциплиной доступа к инструментам

И то и другое **вызывается вручную**. Универсально нужные стандарты должны быть в CLAUDE.md (всегда загружено), а не превращаться в команды (надо вызывать).

3.4.1 Frontmatter SKILL.md — три ручки, которые надо знать

```
---
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
argument-hint: "path to analyze"
---
```

Ручка	Эффект
`context: fork`	Запускает скилл в **изолированном суб-агенте**. Подробный вывод остаётся внутри fork; в основную сессию попадает только финальный результат. Для скиллов, генерящих сотни строк промежуточной работы.
`allowed-tools`	Ограничивает, какие инструменты может использовать скилл. Поставьте `["Read", "Grep", "Glob"]` для read-only безопасности; опускайте опасные инструменты (Bash, Write), когда не нужны.
`argument-hint`	Запрашивает у пользователя ввод, когда скилл вызван без аргументов. **Никаких ручек `require_args` или `interactive: true` не существует** — `argument-hint` и есть ответ.

Эти три ручки активно тестируются. Запомните, что делает каждая, и чего каждая *не* делает.

3.5 Plan mode vs прямое выполнение

	Plan mode	Прямое выполнение
Для чего	Сложных задач, архитектурных решений, нескольких валидных подходов, многофайловых изменений	Хорошо очерченных, понятных одиночных изменений
Примеры	Реструктуризация на микросервисы, миграция библиотеки, затрагивающая 45+ файлов, выбор между подходами интеграции	Добавить условие валидации даты, исправить одиночный баг по понятному стек-трейсу
От какого риска защищает	От дорогой переделки, когда зависимости/решения всплывают поздно	(Избегается оверхед)

Plan mode заслуживается неоднозначностью, не осторожностью. «Всегда используй plan mode» неверно, потому что добавляет оверхед раундов проверки изменениям, в которых нет открытых решений.

Plan + direct можно комбинировать: спланировать миграцию, затем выполнить запланированный подход.

3.6 Суб-агент Explore

Для подробных фаз исследования (картирование 200K-строчной кодовой базы). Работает в изоляции, возвращает сводку в основную сессию. Предотвращает исчерпание окна контекста. Используйте **до** основного расследования, не после; как только шум попал в основную сессию — урон уже нанесён.

3.7 Техники итеративного уточнения

Когда первая попытка модели не та, банк тестирует три шаблона:

- **Конкретные примеры вход/выход** — наиболее эффективны, когда прозовые описания интерпретируются по-разному. 2–3 примера побьют развёрнутую прозу.
- **TDD-итерация** — пишете тесты сначала; итерируете, делясь падениями.
- **Шаблон интервью** — попросите Claude задавать вам вопросы перед реализацией в незнакомой области, чтобы выявить соображения, о которых вы могли не подумать (инвалидация кэша, режимы отказа и т. д.).
- **Одно сообщение vs последовательно** — для **взаимодействующих** проблем — одно подробное сообщение; для **независимых** — последовательная итерация.

3.8 Claude Code в CI/CD

Набор флагов CI:

Флаг	Назначение
`-p` или `--print`	Неинтерактивный режим. Обработать, вывести, выйти. **Без этого джоб зависнет** в ожидании ввода.
`--output-format json`	Выводить как JSON, а не прозу
`--json-schema <file>`	Принудить конкретную JSON-схему к выводу

Использование:

```
claude -p --output-format json --json-schema ./review-schema.json "review this PR"
```

Claude Code использует **стандартные POSIX exit-коды** — ненулевой при сбое. CI напрямую проверяет код выхода.

Выдуманные флаги, которые являются ловушками экзамена: `--batch`, `--json-only`, переменная окружения `CLAUDE_HEADLESS`.

3.8.1 Два вывода из одного вызова

Если вам нужны и машинно-парсимый JSON, и человеческая сводка — **не запускайте дважды**. Запустите один раз с JSON-схемой, затем сгенерируйте сводку из структурированного JSON в вашем CI-скрипте.

3.8.2 Та же сессия Claude, что писала код, не должна его проверять

Self-review слаб — модель удерживает свой контекст рассуждений и предвзята к согласию с собой. Используйте **отдельный экземпляр Claude** без предыдущего контекста для проверки.

3.8.3 Инкрементальные проверки

После первой проверки PR, для последующих пушей **включайте предыдущие находки в контекст** и инструктируйте Claude сообщать только о новых или ещё не устранённых проблемах. Иначе разработчики получат дублирующиеся комментарии.

3.8.4 Контекст для генерации тестов

Передавайте существующие тестовые файлы в контексте, чтобы генерация не предлагала дублирующиеся сценарии. Документируйте стандарты тестирования/фикстуры в CLAUDE.md.

3.9 Ограничение инструментов на уровне настроек

Если инструмент (например, Bash) должен быть запрещён в конкретном репозитории, это ограничение живёт в **конфиге разрешений `allowed-tools` уровня настроек**, а не в инструкции

CLAUDE.md. Структурное ограничение означает, что инструмент не виден модели — пропускать нечего.

3.10 `settings.json` — иерархия и грамматика разрешений

Три расположения, все мерджатся при старте (более позднее перекрывает более раннее):

Файл	Область
`~/.claude/settings.json`	Глобально для пользователя
`<project>/.claude/settings.json`	Проектный, **закоммиченный**
`<project>/.claude/settings.local.json`	Проектный, **личный**, по умолчанию в gitignore

Распространённые ключи:

```
{
  "model": "claude-sonnet-4-6",
  "permissions": {
    "allow": ["Bash(git status)", "Bash(git diff:*)", "Read"],
    "deny": ["Bash(rm:*)", "Write(.env)"],
    "defaultMode": "default"
  },
  "hooks": { "PreToolUse": [...], "Stop": [...] },
  "env": { "DEBUG": "true" }
}
```

Грамматика правил разрешений (используется в settings.json, slash-командах и файлах суб-агентов):

Правило	Совпадает с
`ToolName`	Любым использованием этого инструмента
`Bash(git status)`	Точной командой
`Bash(git diff:*)`	Префиксным совпадением (обратите внимание на двоеточие)
`Read(/etc/**)`	Glob-путём
`WebFetch(domain:github.com)`	Доменным совпадением для фетчеров
`mcp__<server>`	Всеми инструментами одного MCP-сервера
`mcp__<server>__<tool>`	Конкретным MCP-инструментом

Deny побеждает allow на каждом слое. Project-level allow **не** перекрывает user-global deny. Это частая ловушка — пользователи добавляют project allow, ожидая, что тот снимет запрет.

3.11 События хуков в Claude Code

Та же таксономия, что и в Agent SDK (§1.12) — Claude Code это и есть SDK с CLI поверх. settings.json поддерживает:

Событие	Срабатывает
`PreToolUse`	До вызова инструмента
`PostToolUse`	После результата
`UserPromptSubmit`	Пользователь отправил сообщение — можно переписать или впрыснуть контекст
`Stop`	Перед возвратом Claude в idle
`SubagentStop`	Суб-агент завершился
`PreCompact`	До запуска `/compact`
`SessionStart`	Открытие сессии Claude Code
`SessionEnd`	Закрытие сессии Claude Code
`Notification`	Claude собирается показать уведомление

Обработчики хуков — shell-команды. Они получают JSON-payload на stdin и отвечают через:

- Exit `0` — пропустить как есть
- Exit `1` (со stderr) — заблокировать (для `PreToolUse`) или показать предупреждение
- Exit `2` — заблокировать молча
- Напечатать JSON-payload в stdout, чтобы **изменить** значение на лету (например, `{"hookSpecificOutput": {"additionalContext": "..."}}` для `UserPromptSubmit`)

Пример конфигурации хука:

```
{
  "hooks": {
    "PostToolUse": [
      { "matcher": "Write|Edit", "hooks": [{ "type": "command", "command": "pnpm format" }]
    ]
  }
}
```

Ловушка: «Я хочу обязательно запускать `pnpm format` после каждого `Write`». Ответ: `PostToolUse`-хук с `matcher: "Write|Edit"`, **а не** «попросить Claude в CLAUDE.md помнить». Снова вероятностное против детерминированного (§0.6).

3.12 Режимы разрешений

При запуске Claude Code существует четыре режима:

Режим	Поведение
`default`	Запрашивает на каждый явно не разрешённый инструмент
`acceptEdits`	Авто-разрешает правки файлов (Read/Write/Edit), но всё равно спрашивает на Bash и прочее
`plan`	**Только чтение**: нет Write/Edit/Bash; модель производит только план
`bypassPermissions`	Никаких запросов. Опасно; только для доверенной автоматизации

В CI обычно используется `--permission-mode bypassPermissions` (с узким allowlist) или предзаполненные одобрения через настройки. В локальном интерактивном использовании `plan` — это то, на что переключается `/plan`.

Ловушка: «Использовать Claude Code в деплой-джобе и при этом быть безопасным». Неверно: `default` (зависнет на запросах). Неверно: `bypassPermissions` без guardrails. Верно: узкий `allow` в настройках + `bypassPermissions` для остального ИЛИ `acceptEdits`, если shell-доступ не нужен.

3.13 Output styles и statuslines

- **Output styles** (`.claude/output-styles/<name>.md`) — модифицируют *системный промпт самой беседы*, не только форму ответа. Используйте для «explain mode» (подробная педагогика), «concise mode» (без вступлений) или доменных персон.
- **Statuslines** — shell-команда, эмиттящая одну строку, отображаемую внизу TUI; полезна для показа ветки, модели, использования токенов или стоимости.

Output style ≠ slash-команда ≠ скилл ≠ суб-агент. Output style меняет, как Claude ведёт себя *по всей сессии*; остальные вызываются или делегируются.

3.14 Headless-режим — форматы вывода

`-p` / `--print` переключает Claude в headless-режим. `--output-format` принимает:

Значение	Результат
`text` (по умолчанию)	Простой финальный ответ
`json`	Один JSON-объект: `{result, cost_usd, duration_ms, ...}`
`stream-json`	NDJSON-события (один на content-блок / вызов инструмента)

`stream-json` — правильный выбор для CI-дашбордов: каждый вызов инструмента появляется по мере происхождения, можно рендерить инкрементальный статус, и парсинг идёт без отрезания markdown. `json` правилен, когда нужны только финальный ответ и метаданные.

`--input-format stream-json` позволяет пайпать скрипт заранее сформированных сообщений (продвинутое — полезно для воспроизведения структурированного плана).

3.15 `--resume` vs `--continue`

Флаг	Поведение
`--continue` (или `-c`)	Возобновить **самую недавнюю** сессию в текущем проекте — имя не нужно
`--resume <id>`	Возобновить конкретную сессию по ID
`--resume` (без id, интерактивно)	UI-выбор

`--continue` — повседневное «продолжаю с того же места». `--resume` — для cherry-pick'a несвежей сессии. Оба загружают предыдущие результаты инструментов — и §1.6.2 всё ещё применим: не возобновляйтесь в устаревший репозиторий.

3.16 Формат файла slash-команды

Проектная slash-команда в `.claude/commands/<name>.md`:

```
---
description: "Quickly review staged changes"
allowed-tools: ["Read", "Bash(git diff:*)"]
argument-hint: "[optional file path]"
model: claude-haiku-4-5
---
Review the staged changes. Focus on: $ARGUMENTS

If no path was given, review all staged files.
```

Ручка frontmatter	Эффект
`description`	Что видит пользователь в picker'e slash-команд
`allowed-tools`	Грамматика разрешений (§3.10) — сужает доступ без session-wide настройки
`argument-hint`	Заглушка, показанная когда аргументы не введены (та же идея, что и в SKILL.md §3.4.1)
`model`	Опциональное переопределение модели для этой команды

`$ARGUMENTS` подставляется тем, что пользователь набрал после `/<name>`. Вместе эти ручки позволяют одной команде работать на Haiku с узким набором инструментов — правильная форма для дешёвых-и-быстрых процессов вроде быстрых обзоров или извлечений.

ЧАСТЬ 4 — Домен 4: Prompt-инжиниринг и структурированный вывод (20% экзамена)

4.1 Конкретные критерии бьют расплывчатые квалификаторы

Неверно: «будь консервативен», «сообщай только высокоуверенные находки», «будь тщателен», «будь осторожен». Верно: «отмечай комментарии только когда заявленное поведение противоречит фактическому коду». Верно: «сообщай о багах, проблемах безопасности и поломках API; пропускай мелкие стилистические и локальные паттерны». Верно: уровни severity, определённые с **конкретными примерами кода** для каждого уровня.

Самый глубокий шаблон неверного ответа: «перефразируй системный промпт сильнее». Усиление прозы не лечит расплывчатых критериев; переписывание с конкретными якорями — лечит.

4.1.1 Проблема распада доверия

Если у категории находок 70% ложноположительных, разработчики перестают читать весь вывод. **Временно отключите плохую категорию**, пока переписываете её промпт с явными критериями. Доверие асимметрично: легче потерять, чем восстановить.

4.2 Few-shot prompting

Few-shot примеры наиболее эффективны когда:

- Задача связана с **неоднозначностью** (какой инструмент выбрать в пограничном случае)
- Желаемый **формат вывода** конкретен (расположение, проблема, severity, предлагаемое исправление)
- Вход имеет **структурное разнообразие** (научные статьи в формате inline-цитат vs нумерованных ссылок)
- Вы хотите **отличить приемлемые шаблоны от настоящих проблем** (снижение ложноположительных)

Разнообразие лучше объёма. После 12 примеров одного шаблона добавление новых того же шаблона не помогает. Диверсификация для покрытия разных структурных шаблонов — помогает.

Few-shot — **не** правильный инструмент для принуждения (поднимайтесь до хуков для этого).

4.3 Структурированный вывод через tool_use + JSON-схемы

Самый надёжный способ получить вывод, соответствующий схеме — **`tool_use` с JSON-схемой**. Модель вынуждена выдать аргументы, удовлетворяющие схеме. Синтаксические ошибки JSON

исчезают. Запрещённые поля невозможны. Enum-ы ограничены.

Анти-паттерн: prompt-based JSON («верни только JSON, без markdown»). Всегда вероятностно. Кодовые ограждения, вступительная проза, пропущенные скобки — всё возможно.

Анти-паттерн: вытащить JSON-блок из прозы регуляркой. Кошки-мышки.

```
# Верно: tool_use со схемой
tools = [{
    "name": "extract_invoice",
    "input_schema": {
        "type": "object",
        "properties": {
            "vendor": {"type": "string"},
            "total": {"type": "number"},
            "line_items": {"type": "array", "minItems": 1, ...},
            ...
        },
        "required": ["vendor", "total", "line_items"]
    }
}]
```

4.3.1 Что схемы устраняют, а что нет

Схемы устраняют	Схемы не устраняют
Синтаксические ошибки JSON	Семантические ошибки (line items не суммируются в указанный total)
Запрещённые поля	Неправильное размещение значений по полям
Невалидные значения enum	Неправильное-значение-в-правильной-форме
Неправильные типы полей	Галлюцинированный контент

Для семантических проверок добавляйте **парные поля**: `stated_total` + `calculated_total`, затем сравнивайте программно. Или `stated_value` + `verbatim_quote_supporting_value`. Самосогласованность через структуру.

4.4 Дизайн схем

4.4.1 Required vs optional vs nullable

Если поле **может отсутствовать в исходном документе**, отметьте его nullable/optional. **Required-без-null поле принуждает модель к фабрикации.** Это шаблон «номер факса» (D4-003) и шаблон «email» (D4-021).

4.4.2 Шаблон «other» + детали

Enum'ы, которые могут расширяться: `category: enum["A", "B", "C", "other"]` плюс `other_category: string | null`. Когда данные не подходят, модель выбирает `other` и описывает.

Без этого модель вынуждена ошибочно классифицировать.

4.4.3 Первоклассное «unknown»

Если «не могу понять по источнику» — валидный ответ, сделайте его выразимым:

```
{
  "conclusion": {
    "status": "supported" | "refuted" | "insufficient_evidence",
    "details": "..."
  }
}
```

Магические строки («UNKNOWN», «N/A», «various») хрупки — они проходят валидацию схемы как обычные строки, но ломают downstream. Сделайте неопределённость структурной.

4.4.4 Наличие vs значение

Если поле иногда присутствует-но-ноль, а иногда отсутствует — **два поля**: `tip_present: boolean` и `tip_amount: number | null`. Не путайте «0 чаевых» с «нет поля чаевых».

4.4.5 Правильные типы для правильных вещей

Деньги: `number`, с отдельным полем `currency`. Не строки `"$1,234.56"`. Даты: включайте `publication_date` или `as_of_date` на каждой данной точке, которая может меняться со временем. (Иначе временные различия выглядят как противоречия.)

4.4.6 minItems и другие ограничения JSON Schema

Да, JSON Schema может принудить «непустой массив» через `minItems: 1`. Используйте ограничения схемы для простых структурных правил; используйте post-extraction валидацию для cross-field правил.

4.5 Циклы валидации-повтора

Когда извлечение валидируется как JSON, но проваливает семантические проверки (арифметика не сходится, внутренняя несогласованность), повторяйте, **добавив в промпт неудачное извлечение и конкретную ошибку валидации**:

> «У вашего предыдущего извлечения `stated_total: 100, calculated_total: 95`. Line items должны суммироваться в указанный total. Пожалуйста, переизвлеките из этого документа.»

Критическое ограничение: **повторы полезны только для format/structural ошибок, не для отсутствующей информации.** Если данных просто нет в источнике, повтор их не материализует. Правильная реакция — пометить поле null и показать пробел.

4.6 Confidence-оценки некалиброванны

Банк раз за разом тестирует принцип: **не делайте решения зависимыми от модельной оценки уверенности.** Уверенность коррелирует с многословностью, не точностью.

Используйте **структурные проверки** для маршрутизации: валидация схемы, согласованность арифметики, наличие источника, флаг неоднозначных/противоречивых источников.

Для маршрутизации на человеческую проверку калибруйте field-level confidence на **размеченном валидационном наборе**, прежде чем доверять.

4.7 Message Batches API

Свойство	Значение
Стоимость	**50% экономии** против синхронного API
Задержка	До **24 часов** обработки, без SLA
Tool calling	**Не поддерживает многоходовый tool calling** в одном запросе
Корреляция	Поле `custom_id` на каждом запросе; возвращается на ответе

Когда использовать:

- Ночные отчёты
- Еженедельные аудиты
- Ночная генерация тестов
- Любые незамкнутые, толерантные к задержке нагрузки

Когда **не** использовать:

- **Блокирующие pre-merge проверки** (разработчики ждут результат) — 24h-хвост недопустим
- Workflow, требующий многоходового tool calling в одном запросе

4.7.1 SLA-арифметика

Если вам нужен 30-часовой SLA, а batch API может занять 24 часа, можно сабмитить с **окнами в 4 часа** (24h batch + 4h scheduling = 28h, с запасом).

4.7.2 Обработка сбоев в batch

Используйте `custom_id` для идентификации каждого запроса. По завершении batch'а **пересабмитьте только сбойные документы** (с модификациями: например, разбить документы, превысившие лимит контекста). Не перезапускайте весь batch.

Используйте уточнение промпта на **выборке** перед обработкой 10 000 документов. Итерация на выборке из 100 документов дёшева; итерация на полном batch'е дорога.

4.8 Multi-instance и multi-pass review

4.8.1 Ограничения self-review

Модель, только что сгенерировавшая код, удерживает контекст рассуждений — она менее склонна сомневаться в собственных решениях в той же сессии. **Используйте отдельный экземпляр Claude** для проверки.

4.8.2 Multi-pass для больших проверок

Для PR'ов на много файлов: **per-file проходы** для локальных проблем + **отдельный cross-file integration проход** для потоков данных. Избегает разбавления внимания и противоречивых находок (где один и тот же шаблон отмечен в одном файле и одобрен в другом).

Неверные шаблоны:

- «Возьми контекст побольше» — не лечит качество внимания
- «Попроси разработчиков разбивать PR'ы» — перекладывает бремя, не лечит систему
- «Запусти три прохода и помечай только согласованные» — подавит реальные баги, ловимые с перерывами

4.9 Прочие prompt-инжиниринг моменты

- **Стабильные, task-independent инструкции — в системный промпт.** Per-task специфика — в user-сообщение.
- **Префиллите ход ассистента** тегами `<thinking>` (или используйте extended thinking), когда хотите гарантированных пошаговых рассуждений.
- **Сокращайте промпт, удаляя**: пере-формулировки правил, нерелевантный фон, наполнитель («пожалуйста, будьте точны»). **Сохраняйте**: критерии, примеры, ограничения схемы.
- **Температура 0** уменьшает дисперсию, но не лечит неопределённые критерии. Считайте побочной ручкой, не решением надёжности.
- **Seed + temp 0** ещё больше уменьшает дисперсию, но не строгая гарантия между версиями моделей.
- **Extended thinking** меняет задержку на глубину рассуждений. Включайте для тяжёлых рассуждений; не включайте для структурного извлечения и простой классификации, где не помогает.
- **Кэширование промпта** резко снижает стоимость/задержку, когда стабильный 5K+-токенный префикс шерится между многими вызовами. Всегда используйте для этого шаблона.

4.10 Механика prompt caching

Самый большой рычаг стоимости и задержки в продакшене. Помечайте стабильное префиксное содержимое через `cache_control: {"type": "ephemeral"}`, и API сохранит префикс; последующие

запросы, разделяющие его, платят cache-read ставку (~10% стоимости input) за закэшированную часть.

```
{
  "system": [
    {"type": "text", "text": "<long stable system prompt>"},
    {"type": "text", "text": "<tool catalog>", "cache_control": {"type": "ephemeral"}}
  ],
  "messages": [...]
}
```

Правила, которые надо запомнить:

- **Не более 4 cache breakpoint'ов** на запрос. Размещайте на семантических границах (system, tools, предыдущие ходы) — не произвольно.
- **Cache hit требует точного совпадения префикса.** Изменение даже одного токена выше breakpoint'а инвалидирует кэш с этой точки и далее.
- **Порядок имеет значение.** Стабильное идёт первым, волатильное — последним. Breakpoint полезен только если всё *выше* него стабильно.
- **Кэш по (org, model, точному префиксу).** Переключение модели ломает кэш.
- **TTL по умолчанию — 5 минут**; где поддерживается — доступен 1-часовой TTL (другая цена).
- **Что кэшируется:** `system`, `tools`, целые `messages`-блоки. Волатильный финальный user-ход обычно не кэшируется.
- **`usage` возвращает** `cache_creation_input_tokens` (по премиум-ставке записи, ~~25% сверх input) и `cache_read_input_tokens`~~ (10% от input).

Когда вопрос упоминает «у нас 5K токенов стабильного системного промпта и десятки вызовов в минуту» — **кэш и есть фикс**. Когда «у каждого запроса уникальный системный промпт» — кэш не поможет, и неверные ответы как раз размывают это различие.

4.11 Extended thinking

Установите `thinking: {"type": "enabled", "budget_tokens": N}`, и модель производит внутренние рассуждения перед ответом. Свойства:

- `budget_tokens` контролирует, сколько токенов рассуждений разрешено (вы платите за них по output-ставке).
- Thinking-блоки должны сохраняться между ходами (§1.14) — включая `signature`.
- Бета `interleaved-thinking-2025-05-14` позволяет модели думать *между* вызовами инструментов в одном ходе — полезно для многошаговых reasoning-агентов.
- Не включайте thinking для тривиальных задач (извлечение, классификация) — это чистая стоимость.
- Полезно для: математики, многошагового планирования, тяжёлого code-review, неоднозначных решений по политике, нюансного синтеза.
- Thinking-токены засчитываются в `max_tokens`, поэтому поднимайте `max_tokens` соответственно.

Ловушка: «Качество вывода плохое на тяжёлых reasoning-задачах». Неверно: «понизить температуру». Верно: «включить extended thinking» или «увеличить thinking-бюджет».

4.12 Vision и document inputs

Image-блоки принимают base64, URL или `file_id` (Files API):

```
{"type": "image", "source": {"type": "base64", "media_type": "image/png", "data": "..."}}
```

Document (PDF) аналогично:

```
{"type": "document", "source": {"type": "base64", "media_type": "application/pdf", "data": "
```

PDF обрабатываются постранично; модель видит и извлечённый текст, *и* отрендеренные изображения страниц, поэтому графики и таблицы видны. Практические лимиты: ~32 МБ / ~100 страниц на запрос. Для огромных PDF разбивайте по границам страниц и обрабатывайте сфокусированными проходами (§5.1 — потеряно в середине).

4.13 Citations

Установите `citations: {"enabled": true}` на document-блоке, и модель возвращает `citation`-блоки с именами источниковых документов и символьными диапазонами наряду со своей прозой. Это ответ уровня API на вопрос «как доказать, что утверждение модели взято из входа?» — ту же роль, что поля провенанса в схемах (§5.5), но с детерминированным span к источнику.

Совет по дизайну схем: когда citations доступны, требуйте цитату в схеме — не позволяйте модели эмиттить численное утверждение без присоединённого span к источнику.

4.14 Token counting endpoint

Перед отправкой большого запроса вызовите `POST /v1/messages/count_tokens` с тем же payload (минус `max_tokens`). Ответ: `{input_tokens: N}`. Используйте, чтобы:

- Решить, где разместить cache breakpoint
- Решить, нужно ли разбить перед отправкой
- Избежать 400 на переполнении контекста

Не приближайте по длине строки — токенизация нелинейна, особенно для кода и не-латинских скриптов.

ЧАСТЬ 5 — Домен 5: Управление контекстом и надёжность (15% экзамена)

Этот домен мал, но содержит шаблоны с высокой отдачей, применимые везде.

5.1 Эффект «потеряно в середине»

Модели надёжно обращают внимание на **начало** и **конец** длинного входа и могут недо-весить середину. Поэтому:

- Помещайте **сводки ключевых находок в начало** агрегированных входов.
- Используйте **явные заголовки разделов** для облегчения извлечения.
- Когда можно, помещайте **самые важные элементы по краям**.
- Для очень длинных источников (100+ страниц PDF): **разбивайте и обрабатывайте сфокусированными проходами**.

Неверные шаблоны: «Возьми окно побольше» (ухудшает форму кривой), «увеличь max_tokens» (меняет вывод, не внимание ко входу), «вставь весь PDF дважды».

5.2 Обрезка вывода инструментов в источнике

Когда инструмент возвращает 60 полей и важны 4, после нескольких вызовов контекст распухает и модель теряет факты. **Решение — на уровне инструмента**: добавить параметр `fields: [...]` или предоставить slim-вариант `get_customer_summary`. Обрезка на стороне источника — самое эффективное управление контекстом.

Если инструмент изменить нельзя, используйте **`PostToolUse`-хук** для обрезки ответа до того, как модель его увидит.

5.3 Шаблон «case facts» блока

Для длинных, многотемных разговоров: **извлекайте транзакционные факты** (суммы, даты, ID заказов, статусы) в структурированный «case facts» блок, который вставляется в **каждый промпт**, вне суммированной истории. Сводка сжимает прозу; case-facts блок держит числа дословно.

Это предотвращает каноничный режим отказа, когда агент путает $47.99 с $23.50 после суммирования.

5.4 Scratchpad-файлы для длинных сессий

Для исследовательских сессий: пусть агент записывает ключевые находки в scratchpad-файл по ходу, затем ссылается на файл в последующих вопросах. **Scratchpad переживает суммирование.**

Агент перестаёт перечитывать файлы, потому что может перечитать собственные заметки.

Для многоагентных систем с восстановлением после сбоев: **структурированные экспорты состояния агента** (manifests), которые координатор загружает при resume и впрыскивает в промпты агентов.

5.5 Провенанс: маппинги claim → source

Если вы не сохраняете атрибуцию источника **на каждом шаге**, суммирование её уничтожит. Downstream-отчёт выглядит авторитетно, но не аудируем.

Структурное решение:

- Схема вывода каждого суб-агента требует `source_url`, `document_name`, `relevant_excerpt` для каждой претензии.
- Схема вывода агента синтеза требует того же — источники должны распространяться.
- **Required, не nullable.** Если претензию нельзя источниковать — её не следует выпускать как претензию.

Чтобы предотвратить заглушки-источники вроде «various», **сузьте тип поля source** до структурированного объекта `{name, url, publication_date}` с ограничениями формата (URL-шаблон). «various» не пройдёт валидацию.

Для числовых претензий требуйте `verbatim_quote` из источника, поддерживающего число. Пустая цитата → нарушение схемы. Принуждает модель действительно найти источник, а не фабриковать.

5.6 Противоречивые данные источников

Два достоверных источника не согласны (один говорит 30%, другой 45%). Неверно:

- Усреднить их.
- Взять более свежий и отбросить другой.
- Выбрать молча.

Верно: **сохранить оба значения с атрибуцией источника и явно аннотировать конфликт.** Пусть downstream решает. Вывод синтеза должен структурно отличать **установленные находки** от **спорных**.

Когда у одного документа внутренние конфликты (abstract говорит X, рисунок Y), анализирующий суб-агент **завершает анализ с обоими значениями, аннотированными как конфликтующие**. Не эскалировать; не выбирать. Координатор позже решит примирение.

5.7 Временные данные

`2022 = 35%` и `2024 = 52%` — не противоречия, это разные точки времени. Требуйте поле `as_of_date` или `publication_date` на каждой данной точке, которая может меняться. Без него рост-во-времени читается как противоречие.

Для меняющихся фактов (текущий CEO, текущая цена) требуйте, чтобы агент **получал их свежими** из авторитетного инструмента, а не полагался на знания времени обучения.

5.8 Re-grounding vs case-facts блок

Оба сохраняют факты актуальными в длинных сессиях:

- **Case-facts блок** — дёшево, удерживает уже верифицированные факты.
- **Re-grounding** — перечитывает результаты инструментов, потому что лежащие в основе факты могли измениться (статус заказа сдвинулся, запись клиента обновилась).

Используйте re-grounding, когда факты могли измениться; иначе case-facts блок.

5.9 Триггеры эскалации

Уместные причины эскалации:

1. **Клиент явно требует человека.** Уважить немедленно, не пытаться сначала расследовать.
2. **Политика двусмысленна или умалчивает** о конкретном запросе клиента (например, прайс-матч с конкурентом, когда политика говорит только о собственных корректировках).
3. **Невозможность сделать содержательный прогресс** после добросовестных попыток.

Неуместные триггеры эскалации:

- **На основе сентимента.** Раздражение клиента ≠ сложность кейса.
- **Самоотчёт уверенности ниже порога.** LLM-уверенность некалиброванна.
- **Несколько совпадений на поиск клиента.** Спросить дополнительные идентификаторы; не выбирать эвристически и не эскалировать.

Когда клиент раздражён, но проблема в рамках возможностей: подтвердить раздражение, предложить решение, эскалировать только если клиент повторно настаивает на человеке.

5.10 Изоляция данных

Per-customer скоупинг должен быть принуждён **на уровне данных**, не в промптах. Агент получает только факты о текущем `customer_id`, принуждаемые инструментом/сервисом. Промпт-правила («не утекай между клиентами») проваливаются.

5.11 Калибровка уверенности

Для high-volume извлекательных конвейеров:

- Используйте **стратифицированную случайную выборку** высокоуверенных извлечений для измерения частоты ошибок и обнаружения новых паттернов ошибок.
- Валидируйте точность по **типу документа и сегменту полей** — 97% в среднем может скрывать 60% по конкретному типу документа.
- Калибруйте field-level confidence на **размеченном валидационном наборе** перед использованием для маршрутизации.
- Маршрутизируйте сначала низкоуверенные и неоднозначно-источниковые извлечения людям (ограниченная пропускная способность ревьюверов).

5.12 Многотемный контекст

Три отдельных биллинговых вопроса в одной сессии: извлекайте структурированные per-issue данные (ID заказов, суммы, статусы) в отдельный контекстный слой. Не давайте им смешиваться.

Для multi-region агентов (US, EU, AP) впрыскивайте поле `region` в структурированный session-context блок; выражайте политику как region-keyed правила. Агенту не нужно помнить регион; структура его несёт.

5.13 Внутренняя согласованность

Когда вывод синтеза цитирует разные числа в соседних абзацах: **сначала эмитьте структурированную таблицу фактов**, затем нарратив должен цитировать записи из этой таблицы. Несогласованность становится обнаружимой (и предотвращаемой).

5.14 Cost-aware архитектура

Реальные экзаменационные вопросы прячут стоимостные решения. Арифметика, которую надо держать в голове:

- **Output ≈ 5× input.** Многословные ответы — главный драйвер стоимости. Ограничивайте `max_tokens`. Используйте schema-bound выводы (§4.3) — они естественно короткие.
- **Cache reads ≈ 10% от input.** Стабильные префиксы окупаются быстро — точка безубыточности обычно после 1–2 попаданий.
- **Batch = 50% скидка**, но допускает хвост ≤ 24 ч (§4.7).
- **Меньшие модели в 5–10× дешевле.** Используйте Haiku для роутинга инструментов, классификации, простого извлечения. Sonnet для продакшн-рассуждений. Opus только там, где он окупается.
- **Cache + batch комбинируются.** Batch-джобы могут попадать в кэш, если префикс был записан недавно.
- **Обрезка результатов инструментов** (§5.2) — это рычаг стоимости, не только контекста — каждый удержанный токен оплачивается по output-ставке × число ходов, в которых он выживает.

Дерево решений для нового конвейера:

1. Толерантно к задержке (≥ 1 ч)? → Batch.
2. Повторяющийся стабильный префикс (≥ ~1024 токена, ≥ 2 вызова за 5 мин)? → Cache.
3. Per-call извлечение или классификация? → Haiku + `tool_use` со схемой.
4. Тяжёлые рассуждения, неоднозначные данные? → Sonnet/Opus + бюджет extended thinking.

Распространённый шаблон экзамена: «хотим снизить стоимость, не теряя надёжность». Неверные ответы тянутся к «использовать модель поменьше» в одиночку (роняет качество) или «сократить промпт» в одиночку (теряет критерии). Верные ответы стекают: **закэшируйте стабильный префикс, направьте классификации на Haiku, переведите ночной аудит в batch, ограничьте `max_tokens` на болтливом пути.**

5.15 Rate limits и retries

API применяет лимиты RPM, TPM (токены в минуту, отдельно input/output) и concurrent-requests на организацию. При превышении:

- HTTP `429` с заголовком `retry-after` (секунды). Уважать.
- HTTP `529` (overloaded). Откатываться экспоненциально.
- HTTP `5xx`. Повторять с jitter.

Продакшн-политика повторов:

- Повторять **только идемпотентные операции** по умолчанию. SDK Anthropic авто-повторяет безопасные сбои.
- Ограничьте число повторов (3–5), чтобы избежать каскадов.
- Используйте **экспоненциальный backoff с full jitter** — `sleep = random(0, base * 2^n)` — не константный и не линейный.
- Отслеживайте per-tool retry-бюджеты, чтобы хрупкий downstream не сжёг общую квоту.

Для многоагентных конвейеров идемпотентность (§1.8) становится критичной — повторы во время состояний с частичным прогрессом — каноничная причина дублирующих возвратов, дважды отправленных писем и т. п.

5.16 Наблюдаемость

Продакшн-агентам нужны:

- **Структурированные логи на ход**: `request_id`, модель, счётчики input/output токенов, cache hit, вызовы инструментов, `stop_reason`.
- **Cost-телеметрия по сессии и по пользователю.** Стоимости взрываются через длинные сессии, не через одиночные запросы.
- **Tool-call traces** — какой инструмент, очищенные аргументы, задержка, категория ошибки (§2.6).
- **Метрики покрытия для многоагентных конвейеров** — «X% отчётов помечают пробел в покрытии» — сигнал, что декомпозиция координатора нуждается в работе (§1.3.4).
- **Графики confidence vs. accuracy** для обнаружения дрейфа калибровки (§4.6, §5.11).

Естественные точки эмита — события хуков (`PostToolUse` , `Stop` , `SubagentStop` , `SessionEnd`) — держите телеметрию вне промпта модели и вне реализаций инструментов.

ЧАСТЬ 6 — Повторяющиеся шаблоны (грамматика экзамена)

Изучив все домены, начинаешь видеть повторяющиеся формы. Это мета-шаблоны.

6.1 Иерархия силы принуждения (повторно)

Слабейшее → сильнейшее:

1. Усилить язык в системном промпте — **почти всегда неверно**
2. Few-shot примеры — полезны для неоднозначности, не принуждения
3. Описание инструмента — полезно для выбора, не для упорядочивания или политики
4. Ограничение JSON-схемы — устраняет классы ошибок
5. Конфигурация `tool_choice` — устраняет «инструмент не вызван»
6. `PostToolUse` хук — детерминированная нормализация данных, обрезка вывода
7. `PreToolUse` хук — детерминированное блокирование, предусловия, политики
8. Логика на уровне инструмента — валидация входа, идемпотентность, двухфазные шаблоны

Когда вопрос описывает «X происходит в N% случаев» и спрашивает решение, поднимайтесь по лестнице. Правильный ответ обычно — 4-й уровень и выше.

6.2 «Лечите первопричину, а не симптом»

Авторы экзамена это любят. Распространённые пары симптом-vs-первопричина:

Симптом (неверный ответ)	Первопричина (верный ответ)
«Вывод неправильного формата»	Использовать `tool_use` со схемой
«Агент пропускает обязательный шаг»	`PreToolUse` хук для принуждения предусловия
«Агент фабрикует поля»	Сделать поле nullable в схеме
«Агент выбирает неправильный инструмент»	Улучшить конкретность описания, затем сократить число инструментов
«Отчёты упускают подтемы»	Декомпозиция координатора слишком узка
«Суб-агенты возвращают поверхностные результаты»	Они не наследуют контекст — передавайте явно
«Длинный обзор PR непоследователен»	Per-file проходы + integration проход (разбавление внимания)
«Агент путает даты между системами»	`PostToolUse` хук нормализует timestamps
«Агент создаёт дублирующие тикеты»	Требовать поиск перед созданием + идемпотентность
«Конфликтующие числа проскальзывают»	Парные поля (stated_total + calculated_total)
«Источники теряются при синтезе»	Required source-поле на каждом слое схемы
«Распухание контекста в длинных сессиях»	Обрезка в источнике + case-facts блок + scratchpad

6.3 Языковые шаблоны отвлекающих вариантов

Банк использует шаблонные фразы на неверных вариантах. Распознавать их — половина дела:

- **«It's a commonly proposed patch that reduces the surface symptom without fixing the root cause.»** → симптом-фикс
- **«The tempting shortcut; works until you hit an edge case that makes the real failure visible.»** → выглядит легко, ломается под нагрузкой
- **«Works in the easy case and leaves the hard cases silently broken.»** → слишком узкое правило
- **«Buys short-term quiet at the cost of hiding a bug that will resurface under load.»** → маскирует проблему
- **«Treats a reliability concern as a prompting concern, which is exactly the framing the guidance cautions against.»** → промпт-заплатка на структурную проблему
- **«Looks plausible at first glance but creates its own failure modes you'd later have to work around.»** → вводит новые проблемы
- **«Feels rigorous but doesn't give the agent the information it needs to recover well.»** → скрывает информацию от downstream
- **«It feels like the familiar lever, but it doesn't address the underlying structural issue.»** → тянется к привычному инструменту, который не подходит
- **«It is superficially attractive and ignores the layer where the actual control point lives.»** → не тот слой

- **«Sounds safe, but the guidance explicitly points elsewhere for this shape of problem.»** → есть документированный паттерн, это не он

Когда видите одну из этих фраз на варианте — этот вариант неверен. Используйте, чтобы быстро сужать поле.

6.4 Анти-паттерн «возьми модель/окно побольше»

Почти всегда неверно. Означает, что отвечающий не понимает фактический механизм. Большее не лечит:

- Качество внимания / потеряно в середине
- Вероятностное vs детерминированное принуждение
- Расплывчатые критерии
- Устаревшие результаты инструментов
- Декомпозицию координатора
- Неоднозначность выбора инструментов

6.5 Анти-паттерн «скажи модели быть осторожнее»

Всегда неверно. Модель — не человек; у неё нет осторожного и неосторожного режимов. Решение структурное, не мотивационное.

ЧАСТЬ 7 — Четыре промаха из вашего пробного экзамена

Закрепляем то, что споткнуло именно вас.

D1-001 — Агентный цикл не должен проверять текст ассистента

Вы выбрали A («системный промпт не инструктирует»). Верно: B («проверка на текстовое содержание ассистента — ненадёжный индикатор»).

Почему: Модель может выдавать пояснительный текст *между* вызовами инструментов. Текстовое содержание — не сигнал завершения; `stop_reason == "end_turn"` — да. (См. §0.5 и §1.1.1.)

D3-026 — Возобновление устаревшей сессии

Вы выбрали C (перечитать каждый файл). Верно: A (начать новую сессию и впрыснуть структурированную сводку).

Почему: Возобновление распространяет устаревшие результаты инструментов из предыдущих ходов. Рассуждения построены на этих результатах. Перечитывание текущих файлов не отменяет предыдущие выводы модели. Выбросьте сессию; перенесите только чистую сводку. (См. §1.6.2.)

D3-028 — Скилл, запрашивающий аргументы

Вы выбрали D (`require_args: true`). Верно: C (`argument-hint: "..."`).

Почему: `require_args` и `interactive: true` не существуют в frontmatter SKILL.md. Реальный механизм — `argument-hint` — Claude Code использует подсказку, чтобы запросить пользователя, когда скилл вызван без ввода. Три ручки frontmatter SKILL.md, которые надо запомнить: `context: fork` , `allowed-tools` , `argument-hint` . (См. §3.4.1.)

D4-033 — Прослеживаемость batch'a

Вы выбрали A (индекс в batch'e). Верно: C (`custom_id` на каждом запросе).

Почему: Порядок batch'a — не гарантированный механизм корреляции. `custom_id` — **спроектированное** поле корреляции; оно возвращается на ответах без изменений. Поставьте его равным user_id (или конкатенации, которую можно инвертировать), и корреляция тривиальна. (См. §4.7.)

ЧАСТЬ 8 — Шпаргалки для быстрого поиска

8.1 Файлы и расположения

Путь	Область	Назначение
`~/.claude/CLAUDE.md`	Пользовательская	Личные предпочтения, не шерится
`<project>/CLAUDE.md` или `.claude/CLAUDE.md`	Проектная	Командные стандарты, версионируется
`<subdir>/CLAUDE.md`	Каталоговая	Каталого-специфические конвенции
`.claude/rules/<topic>.md`	Проектная	Условные правила с YAML `paths`-glob frontmatter
`.claude/commands/<name>.md`	Проектная	Проектные slash-команды
`~/.claude/commands/<name>.md`	Пользовательская	Личные slash-команды
`.claude/skills/<name>/SKILL.md`	Проектная	Проектные скиллы с frontmatter
`~/.claude/skills/<name>/SKILL.md`	Пользовательская	Личные скиллы
`<project>/.mcp.json`	Проектная	Общая конфигурация MCP-сервера
`~/.claude.json`	Пользовательская	Личные MCP-серверы

8.2 Frontmatter SKILL.md

Ручка	Эффект
`context: fork`	Запуск в изолированном суб-агенте; в основную сессию попадает только финальный вывод
`allowed-tools: [...]`	Ограничивает доступ к инструментам
`argument-hint: "..."`	Запрашивает пользователя при вызове без аргументов

(Нет `require_args`, нет `interactive: true`, нет `quiet: true`, нет `return_only`, нет `max_tokens`, нет `readonly`.)

8.3 Флаги Claude Code CLI для CI

Флаг	Назначение
`-p` / `--print`	Неинтерактивный режим (обязательно в CI)
`--output-format json`	JSON-вывод
`--json-schema <file>`	Принудить схему к JSON-выводу

Exit-код: стандартный POSIX, ненулевой при сбое.

(Нет `--batch`, `--json-only`, `CLAUDE_HEADLESS`.)

8.4 Значения `tool_choice`

Значение	Смысл
`"auto"`	Модель может вызвать инструмент или вернуть текст (по умолчанию)
`"any"`	Модель должна вызвать какой-то инструмент, выбирает сама
`{"type": "tool", "name": "X"}`	Модель должна вызвать конкретно инструмент X

(Нет `"required"`.)

8.5 Значения `stop_reason`

Значение	Действие цикла
`tool_use`	Выполнить, добавить результат, зациклить
`end_turn`	Готово, вернуть пользователю
`max_tokens`	Обработать (суммировать частичный, показать усечение)

8.6 Хуки

Хук	Когда	Для чего
`PreToolUse`	До выполнения инструмента	Блокировка, изменение, шлюз предусловий, политика
`PostToolUse`	После возврата инструмента	Нормализация, обрезка, логирование, цепочка

8.7 Управление сессиями

Рычаг	Когда
`--resume <name>`	Продолжить конкретную сессию, контекст ещё актуален
`fork_session`	Ветвление от общей базы, сравнение независимых путей
`/compact`	Уменьшить контекст, **факты ещё актуальны**
Начать заново + сводка	**Факты устарели** (код изменился, мир изменился)

8.8 Метаданные ошибок инструмента

Поле	Назначение
`isError: true`	Стандартный флаг MCP
`errorCategory`	`transient` / `validation` / `permission` / `business`
`isRetryable`	Булево, **явно**
Человекочитаемое сообщение	Для коммуникации агента

8.9 Дефолты дизайна схем

- Поле может отсутствовать в источнике → **nullable** (предотвращает фабрикацию)
- Категория с возможными новыми значениями → enum **+ "other"** + nullable detail-строка
- Нужна cross-field семантическая проверка → **парные поля** (stated_X + calculated_X)
- Неопределённость — валидный ответ → enum **включающий** `"insufficient_evidence"` или `"unclear"`
- Деньги → `number`, отдельное поле `currency`
- Меняющиеся со временем данные → требуют `as_of_date` / `publication_date`
- Требуется провенанс → required `{name, url, publication_date}` с URL-шаблоном

8.10 Решение: plan mode vs direct

Plan	Direct
Multi-file, multi-service, архитектурные решения	Одиночный файл, хорошо очерчено, понятный фикс
Несколько валидных подходов	Решённый подход
Миграция библиотеки на 45+ файлов	Добавить условие валидации даты
Реструктуризация на микросервисы	Фикс по понятному стек-трейсу

8.11 Решение: batch API vs синхронный

Batch	Синхронный
Ночные отчёты	Pre-merge проверки
Еженедельные аудиты	Интерактивный UX
Толерантно к задержке (≤ 24h)	Блокирующие процессы
Single-turn извлечение	Multi-turn tool calling
Чувствительно к стоимости (50% экономии)	Чувствительно к задержке

8.12 Решение: какая примитива Claude Code

Нужно	Использовать
Always-on командные стандарты	Project CLAUDE.md
Стандарты по типу/шаблону файла	`.claude/rules/` с glob `paths`
Стандарты по каталогу	Subdirectory CLAUDE.md
Личные предпочтения	`~/.claude/CLAUDE.md`
Переиспользуемый командный workflow	Проектная slash-команда или скилл
Скилл с подробным выводом	Скилл с `context: fork`
Безопасный read-only скилл	Скилл с ограниченными `allowed-tools`
Условные правила в CI	`.claude/rules/` с `paths`
Общая интеграция с бэкендом	`.mcp.json` с env-var секретами
Личный экспериментальный инструмент	`~/.claude.json`

8.13 События хуков (Agent SDK и Claude Code)

Событие	Для чего
`PreToolUse`	Блокировка / изменение / шлюз предусловий
`PostToolUse`	Нормализация / обрезка / лог / цепочка
`UserPromptSubmit`	Впрыснуть контекст, очистить, прицепить case-facts
`Stop` / `SubagentStop`	Финальная QA, сохранить scratchpad
`PreCompact`	Закрепить факты must-keep
`SessionStart` / `SessionEnd`	Загрузить дефолты, слить телеметрию
`Notification`	Кастомные оповещения

8.14 Режимы разрешений

Режим	Поведение
`default`	Запрашивает на неразрешённые инструменты
`acceptEdits`	Авто-разрешает правки файлов, спрашивает на остальное
`plan`	Только чтение; производит план
`bypassPermissions`	Без запросов (только CI, с узким allowlist)

8.15 Headless-форматы вывода

Формат	Когда
`text`	Человекочитаемое
`json`	Финальный ответ + метаданные одним объектом
`stream-json`	Per-event NDJSON; правильно для дашбордов

8.16 Транспорты и примитивы MCP

Транспорт	Когда
`stdio`	Локальный dev, single user
`sse`	Удалённый, долгоживущий, стриминг
`http`	Современный удалённый дефолт; поддерживает stateless

Примитив	Единица переиспользования — это
Tool	Действие
Resource	Окружающий контент
Prompt	Переиспользуемый шаблон промпта (slash-команда)

8.17 Грамматика правил разрешений

Правило	Совпадает с
`Bash(git status)`	Точной командой
`Bash(git diff:*)`	Префиксом
`Read(/etc/**)`	Glob-путём
`WebFetch(domain:x.com)`	Доменом
`mcp__server`	Всеми инструментами этого сервера
`mcp__server__tool`	Конкретным MCP-инструментом

Deny побеждает allow на каждом слое.

8.18 Рычаги стоимости

Рычаг	Эффект
Cache hit	~10% от input на закэшированном префиксе
Cache write	~125% от input (один раз)
Batch API	50% скидка, хвост ≤ 24 ч
Haiku vs Sonnet	в 5–10× дешевле для роутинга/классификации
`max_tokens` cap	Пропорциональная экономия на output
Trim вывода инструмента	Экономит output × число ходов выживания

8.19 Типы блоков в Messages API

Блок	Где	Заметки
`text`	оба	Простой текст
`image`	user	base64, URL или `file_id`
`document`	user	PDF — постранично
`tool_use`	assistant	Имеет `id`, `name`, `input`
`tool_result`	**user**	Отвечает на `tool_use`
`thinking`	assistant	Сохраняйте `signature` дословно

8.20 События стрима

Событие	Смысл
`message_start`	Каркас ответа
`content_block_start`	Открывается новый блок
`content_block_delta`	Инкрементальный контент (`input_json_delta` для инструментов)
`content_block_stop`	Блок полный — здесь исполнять инструмент
`message_delta`	Финальные `stop_reason` / `usage`
`message_stop`	Конец стрима

ЧАСТЬ 9 — Финальная ментальная модель

Если бы всё в этом руководстве свернуть в одно предложение, оно было бы таким:

> **Модель вероятностна. Систему вокруг модели делает её надёжной. В сомнительных случаях выталкивайте правило наружу — в хук, в схему, в проверку на уровне инструмента, в структурированное поле — пока сбой не станет невозможным по построению.**

Каждый домен — следствие:

- **Домен 1 (Агентная архитектура)** — цикл должен управляться `stop_reason`; детерминированное принуждение (хуки, предусловия) бьёт промпт-инструкции; coordinator-routed многоагентные системы с явной передачей контекста и параллельными `Task`-вызовами.
- **Домен 2 (Инструменты и MCP)** — описания инструментов управляют выбором; структурированные ошибки управляют восстановлением; число инструментов и именование управляют точностью; секреты через env vars; ресурсы для окружающей видимости.
- **Домен 3 (Claude Code)** — трёхуровневая иерархия CLAUDE.md, которая наслаивается; `.claude/rules/` для path-условных правил; скиллы для подробной изолированной работы; plan mode заслуженный неоднозначностью; CI через `-p` + JSON-схема.
- **Домен 4 (Промпты и схемы)** — явные критерии вместо расплывчатых квалификаторов; few-shot для неоднозначности; `tool_use` + JSON-схема для гарантированной структуры; nullable для «может отсутствовать»; парные поля для самосогласованности; `custom_id` для batch-корреляции.
- **Домен 5 (Контекст и надёжность)** — case-facts блоки, scratchpad'ы, обрезка инструментов на стороне источника; провенанс как required schema-поле на каждом слое; конфликты аннотируются, а не разрешаются молча; временные данные с явными датами; «потеряно в середине» смягчается структурой и разбивкой.

Прочтите этот документ дважды. Затем пройдите практические экзамены. Затем перечитайте разделы по тому, что промахнули.

Удачи.